Workflow Automation using Deluge – Insurance Claim Automation Suite

Aim

To design and build an automation layer in the Insurance Claim Request & Processing Management application on Zoho Creator using the eight Workflow options – Form Workflows, Schedules, Approvals, Payments, Blueprints, Batch Workflows, Report Workflows and Functions – so that claim acknowledgement, document follow-up, high-value approval, settlement payout, lifecycle control, nightly reconciliation, recalculation of the net payable amount and settlement-letter generation run automatically with minimal manual intervention.

Procedure

Step 1: Open the Insurance Claim Request & Processing Management application in Zoho Creator and go to the **Workflow** tab, which lists the sub-tabs Form workflows, Schedules, Approvals, Payments, Blueprints, Batch workflows and Functions.

Step 2: Form Workflow: create a new workflow, choose "On a form event", select the Claim Request Form, set Record Event = *Created* and Form Event = *Successful form submission*, name it **On_Claim_Submit_Workflow**, and add a Send Mail action to acknowledge the claim.

Step 3: Schedule: under the Schedules tab create **Daily_Missing_Document_Reminder_Schedule**, set the date and time (10:00 AM daily), and add the action that mails claimants whose documents are still missing.

Step 4: Approval: create **HighValue_Claim_Approval** as an approval workflow, add a Level 1 Approver, and under both the Approve and Reject branches add a Send Mail action to ${approver.submitter} so the submitter is told the outcome.

Step 5: Blueprint: open Blueprints, create **Claim_Lifecycle_Blueprint** on the claim form and define the stages Submitted → Document Verification → Investigation → Approved → Settled / Rejected, followed by the transitions between them.

Step 6: Batch Workflow: create **Bulk_Surveyor_Reconciliation_Batch** ("On batches of records"), set "Execute for each batch block of" to 100 records, and write the Deluge action to run for each record in a block. Run the script using *Execute* to test it.

Step 7: Payments, Report Workflow and Functions: configure **Claim_Settlement_Payout_Workflow** (payment activity, trigger on Status = Approved), **OnEdit_Recalculate_Settlement_Workflow** (report edit event, fires only when Depreciation Percent or Estimated Repair Cost changes) and the reusable Deluge function **Generate_Settlement_Letter_Function**.

Step 8: Verify every workflow from its sub-tab by checking the trigger, the number of actions and the Enabled/Disabled status.

241401084

Workflow Mapping

Workflow option	Workflow name	Trigger / frequency
Form Workflows	On_Claim_Submit_Workflow	On form submission (Created)
Schedules	Daily_Missing_Document_Reminder_Schedule	Daily at 10:00 AM
Approvals	HighValue_Claim_Approval	Net Payable Amount above threshold
Payments	Claim_Settlement_Payout_Workflow	Status changes to Approved
Blueprints	Claim_Lifecycle_Blueprint	Every claim record, five stages
Batch Workflows	Bulk_Surveyor_Reconciliation_Batch	Nightly, batches of 100 records
Report Workflows	OnEdit_Recalculate_Settlement_Workflow	Edit of Depreciation % / Repair Cost
Functions	Generate_Settlement_Letter_Function	Status = Settled, or manual button

2140108

Output

The automation was configured and verified across the different modules available under the Workflow tab of the application, as shown below.

1. Form Workflow – On_Claim_Submit_Workflow

A form workflow named **On_Claim_Submit_Workflow** was created on the Insurance Claim Request Form, triggered on *Created* → *Successful form submission*. Under "Execute without condition" a Send Mail action is configured, so an acknowledgement is sent automatically the moment a new claim is submitted.

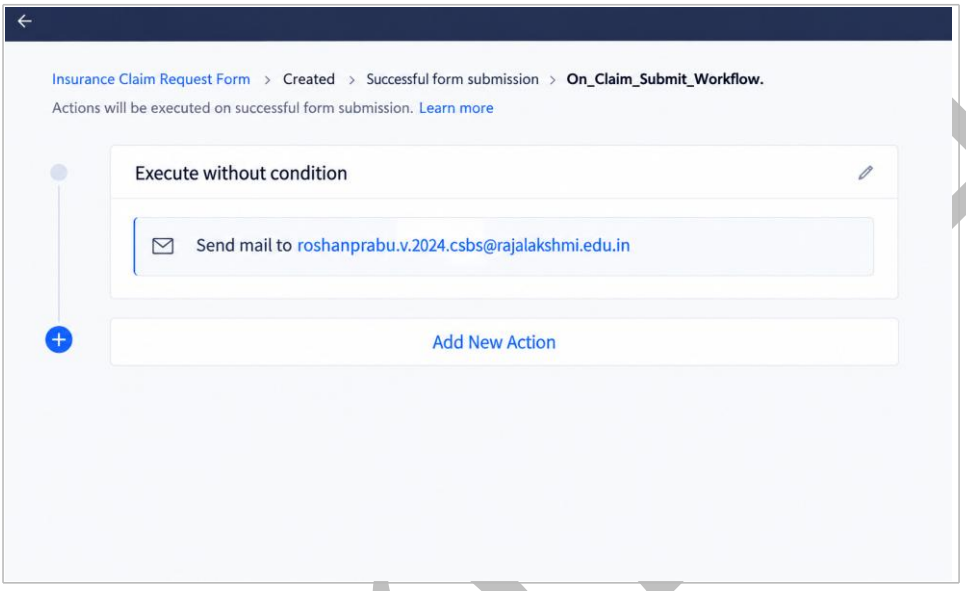


Fig. 1 – On_Claim_Submit_Workflow with its Send Mail action

2. Schedule – Daily_Missing_Document_Reminder_Schedule

A custom schedule named **Daily_Missing_Document_Reminder_Schedule** was created to run on 12-Oct-2026 at 10:00:30, with the "Add New Action" block available for the reminder email to claimants whose required documents are still missing. Because it is time-based, it runs without any form or user event.

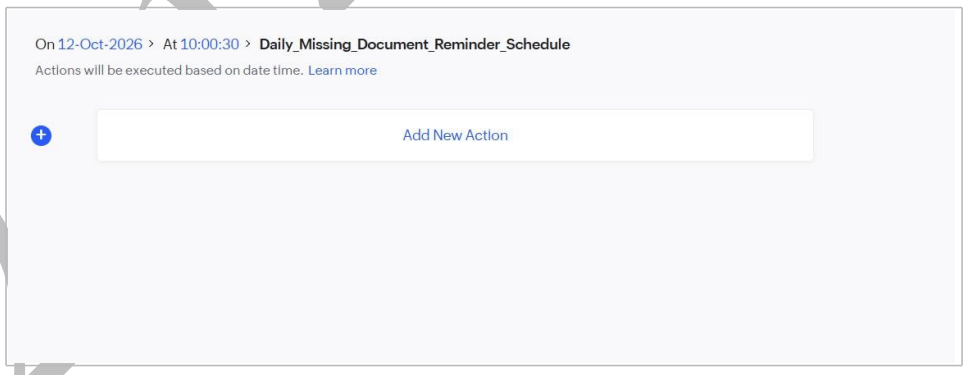


Fig. 2 – Schedule configured with a date and time

3. Approval – HighValue_Claim_Approval

The approval workflow for the Insurance Claim Request Form has a Level 1 Approver (CEO, acting as senior management). Both the **Approve** and **Reject** branches send an email to ${approver.submitter}, so the claimant-side submitter is informed of the decision. The right-hand panel lists the approval events: On Start, Approve and Reject.

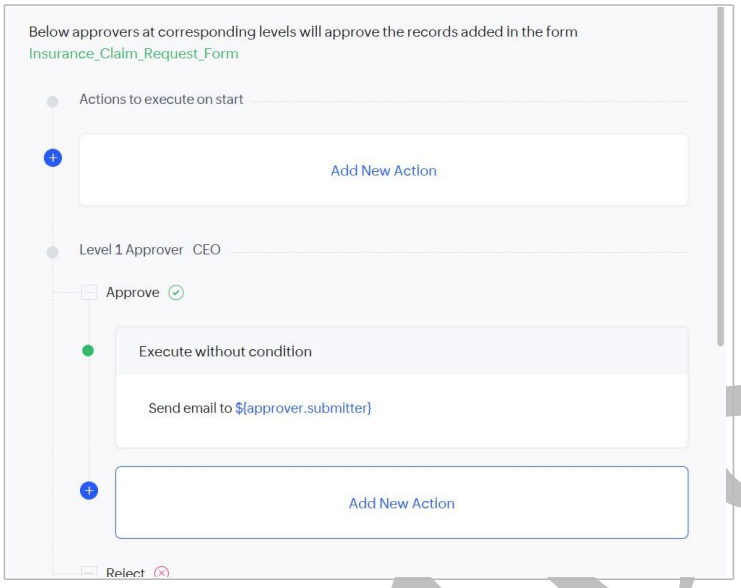


Fig. 3 – Level 1 Approver with Approve and Reject actions

4. Blueprint – Claim_Lifecycle_Blueprint

The blueprint **Claim_Lifecycle_Blueprint** was built on Claim_Request_Form. The canvas shows the lifecycle: Start ("Submitted Claim has been received") → Document Verification → Investigation → Approved → Rejected / Settled. Transitions between these stages are what enforce the controlled, role-based claim process.

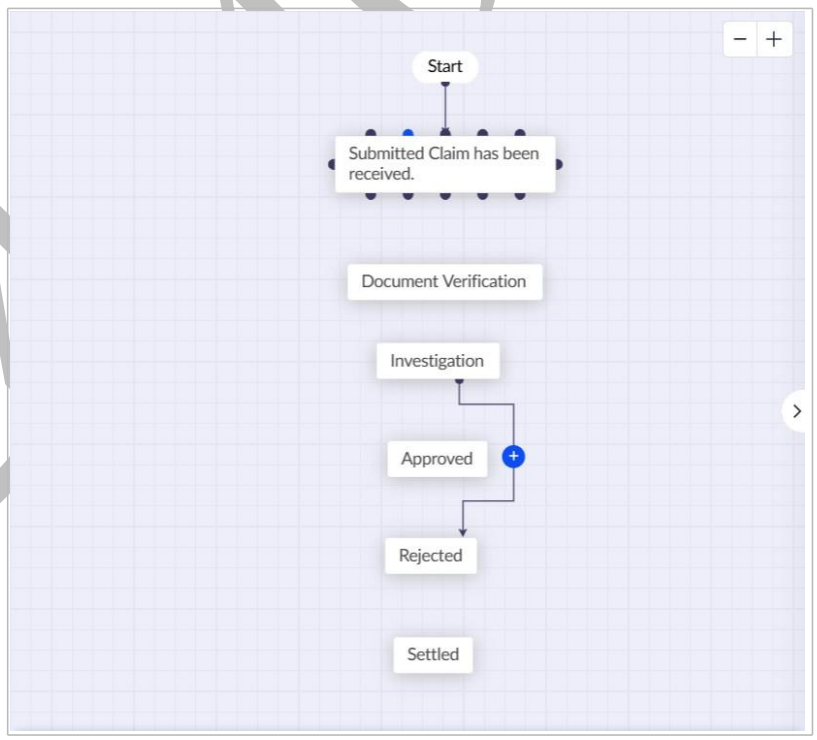


Fig. 4 – Blueprint canvas showing the claim lifecycle stages

In the Workflow → Blueprints tab, the blueprint is listed under "Forms with Blueprints" for the Insurance Claim Request Form with its status shown as **Enabled**.

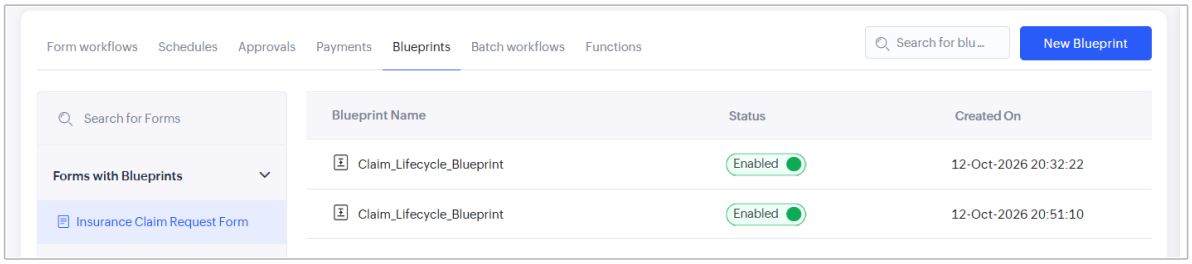


Fig. 5 – Claim_Lifecycle_Blueprint listed as Enabled

5. Batch Workflow – Bulk_Surveyor_Reconciliation_Batch

The batch workflow **Bulk_Surveyor_Reconciliation_Batch** executes for each batch block of **100 records**, so a large claims backlog is processed in controlled blocks rather than all at once. The "Action to execute for each record in a block" is a Deluge script using `sendmail`. Running the script with *Execute* returned "Executed Successfully" in 0.057 seconds, confirming the Deluge action runs correctly.

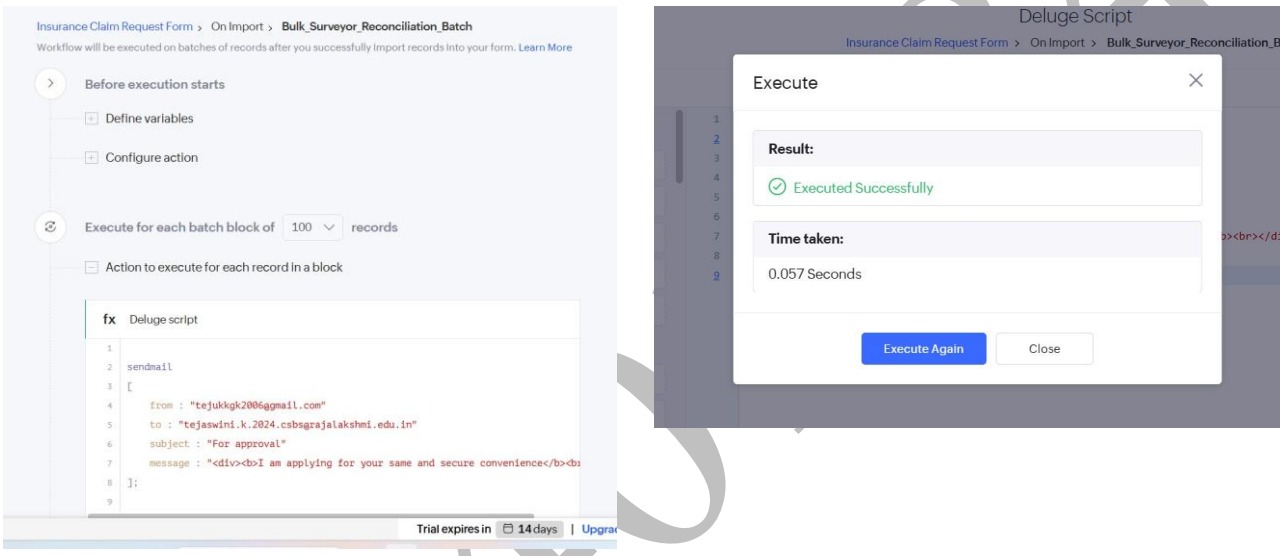


Fig. 6 – Batch block size of 100 with the Deluge action (left) and successful execution result (right)

6. Payments, Report Workflow and Function

- **Claim_Settlement_Payout_Workflow (Payments):** triggered when a claim's Status becomes "Approved"; initiates the payout through the payment/bank-transfer connector, sets Settlement_Status to "Paid" and stores the transaction reference, and retries the status check up to 3 times before flagging failure to finance.
- **OnEdit_Recalculate_Settlement_Workflow (Report Workflow):** triggered on Edit of a record in Claim_Request_Report and restricted to changes in Depreciation Percent or Estimated Repair Cost, so Net_Payable_Amount is recalculated without resubmitting the form.
- **Generate_Settlement_Letter_Function (Function):** a reusable Deluge function that validates Claimant Name and Net Payable Amount, generates the settlement-letter PDF and emails it to the claimant; called automatically when Status = Settled and by the "Resend Settlement Letter" button.

Result

The Insurance Claim Automation Suite was successfully designed and implemented in Zoho Creator using all eight Workflow options. A form workflow acknowledges every new claim, a daily schedule follows up on missing documents, an approval workflow routes high-value claims to senior management, a blueprint enforces the claim lifecycle from Submitted to Settled/Rejected, and a batch workflow processes claims in blocks of 100 records for reconciliation. Together with the payment, report-edit and function workflows, the claims process runs from submission to settlement with minimal manual effort.

Because each workflow is scoped to a single event – record creation, an approval decision, a stage transition, an edit or a clock trigger – the automation stays predictable, and new actions such as SMS alerts or payment reminders can be added to any block without disturbing the others.

Learning Outcomes

- **Choosing the right trigger:** understood the Zoho Creator workflow categories – form events, schedules, approvals, payments, blueprints, batches, report events and functions – and which fits a given requirement.
- **Record vs. form events:** combined Record Events (Created, Edited, Created or Edited, Deleted) with Form Events such as Successful form submission to control exactly when a workflow fires.
- **Deluge scripting:** wrote Deluge actions such as sendmail and bound dynamic values like ${approver.submitter} into them; tested a script using the Execute dialog.
- **Approval-based automation:** configured an approver with separate Approve and Reject branches, each with its own notification.
- **Blueprints:** modelled a claim lifecycle as stages and transitions to give a controlled, auditable process.
- **Batch processing:** used a configurable batch size (100 records) to avoid timeouts on large data sets.
- **Verification:** checked each automation from the Workflow tab for trigger, number of actions and Enabled/Disabled status before relying on it.

Overall, the exercise showed how low-code, event-driven rules can replace repetitive manual work – acknowledgements, reminders, approvals, reconciliation and letter generation – while keeping the logic transparent and easy to maintain.

241401084